

"es un entero"
"es un entero "

ESTRUCTURAS ITERATIVAS

LOOP {FORMA}*. Expresa una construcción iterativa que cicla continuamente, a menos que se le indique explícitamente que pare, evaluando las formas en secuencia. La forma de indicar la terminación del ciclo es mediante RETURN. En general la evaluación de la sentencia RETURN provocará la salida de cualquier sentencia iterativa. Ejemplo,

```
> (setq x 1)
1
> (loop
  (print x)
  (setq x (+ x 1))
  (if (= x 20) (return)))
) ; end loop
Imprime del 1 al 19 y sale con un NIL.
```

DO ({PARAM}* / {(PARAM VALOR)}* / {(PARAM VALOR INCRE)}*)
(TEST-EVALUACION {FORMAS}*) {FORMAS}*

La estructura DO tiene tres partes: lista de parámetros, test de final y cuerpo. La lista de parámetros liga las variables con sus valores iniciales y en cada ciclo se asignan nuevos valores dependiendo de la función de incremento indicada. En la estructura DO, primero se activa la ligadura de los parámetros y después se evalúa el test de final, y si no se cumple se ejecuta el cuerpo.

Esta estructura podemos encontrarle semejanza con la de while ... do utilizada en otros lenguajes.

```
(DO ((var1 inic1 [paso1]))...
  (varn inicn [pason]))          ; Asignación de variables en paralelo
(test-final resultado)          ; Test-final se evalúa antes que cuerpo
  declaraciones
  cuerpo)
```

Ejemplos: Función exponente mn

```
> (setq m 3 n 4)
> (do ((resultado 1) (exponente n))          ; asignación de parámetros
  ; asigna 1 a resultado y n a exponente
  ((zerop exponente) resultado)              ; test de final
                                          ; si exponente es 0, muestra resultado,
                                          ; sino sigue con la próxima sentencia.
  (setq resultado (* m resultado))           ; asigna el producto a resultado
  (setq exponente (- exponente 1)) )         ; resta 1 a exponente, vuelve al
81                                           ; test.
```

```
> (do ((resultado 1 (* m resultado))
  (exponente n (1- exponente)))
  ((zerop exponente) resultado) )
```

81

Otra forma con el test al final.

Ejemplos:

```
> (setq lista '(1 2 3 4))
```

```
> (do ((l lista (cdr l))
      (resultado nil (cons (car l) resultado)))
    ((null l) resultado) )
(4 3 2 1)
```

Iteración	l	resultado
1	(1 2 3 4)	nil
2	(2 3 4)	(1)
3	(3 4)	(2 1)
4	(4)	(3 2 1)
5	()	(4 3 2 1)

DOTIMES permite escribir procedimientos sencillos con iteración controlada por un contador. Ejecuta el cuerpo un número de veces determinado.

```
(DOTIMES (var forma-limite-superior [forma-resultado]) cuerpo)
```

Cuando comienza el ciclo se evalúa la forma límite superior, produciendo un valor n. Entonces desde el valor 0 incrementándose en uno hasta n-1, se asigna a la variable var.

Para cada valor de la variable se ejecuta el cuerpo y al final, la ligadura con la variable se elimina y se ejecuta la forma resultada dando valor al DOTIMES. Si no tiene forma-resultado, finaliza con NIL.

```
> (dotimes (cont 4) (print (* cont cont))
0
1
4
9
NIL
```

```
> (dotimes (x 5 '(fin)) (print (list x x))
(0 0)
(1 1)
(2 2)
(3 3)
(4 4)
Fin
```

DOLIST. asigna a VAR el primer elemento de la lista L; evalúa S1,..., SN para cada valor de VAR; si L no tiene más elementos, devuelve resultado; en otro caso, le asigna a VAR el siguiente elemento de L e itera el proceso. Si no tiene el valor de resultado, finaliza con NIL.

```
(DOLIST (VAR L [resultado])
      S1, ..., SN)
```

```
> (dolist (x '(a b c))
  (print x) )
```

```
A
B
C
NIL
```

FUNCIONES ITERATIVAS PREDEFINIDAS

(MAPCAR fn lista1 ... listan)

Va aplicando la función fn a los sucesivos car's de las listas.

Devuelve una lista con los resultados de las llamadas a la función.

Utiliza LIST para construir el resultado.

```
> (mapcar '+ '(7 8 9) '(1 2 3))      ==> (8 10 12)
```

```
> (mapcar 'oddp '(7 8 9))           ==> (T NIL T)
```

```
> (mapcar 'atom '(A (B) 3))          ==> (T NIL T)
```

(MAPLIST fn lista1 ... listan)

Se aplica la función fn a los CDR's sucesivos de las listas.

Devuelve una lista con los resultados de las llamadas a la función.

Utiliza LIST para construir el resultado.

```
> (maplist '+ '(7 8 9) '(1 2 3))     ==> (8 10 12) (10 12) (12)
```

```
> (maplist 'cons '(1 2 3) '(a b c))  ==> (((1 2 3) A B C) ((2 3) B C) ((3) C))
```

ENTRADA/SALIDA SIMPLE

Lisp tiene primitivas de funciones de entrada/salida. Las más comúnmente utilizadas son:

READ, PRINT y FORMAT, aunque también veremos otras.

Los dispositivos por defecto son, el de entrada el teclado y salida la pantalla, pero se puede redireccionar a otros dispositivos. Es decir, todas estas funciones toman un argumento opcional llamado input-stream u output-stream.

Si este valor no es suministrado o es nil, se tomará por defecto el valor contenido en la variable *standard-input* que será teclado o pantalla respectivamente. Si el valor es t se usará el valor de la variable *terminal-io*.

Función de entrada

- READ, lee un objeto Lisp del teclado y devuelve dicho objeto. Esta función detiene la ejecución del programa mientras no se termine con un RETURN. Sintácticamente tendríamos (READ &optional stream)

Ejemplos:

```
> (read)
```

```
35
```

```
35
```

```
> (setq a (read))
```

```
HOLA
```

```
HOLA
```

```
> a
```

```
Hola
```

Funciones de salida

PRINT, toma un objeto Lisp como argumento y lo escribe en una nueva línea con un blanco por detrás (introduce <Return> y blanco después del objeto). Escribe los objetos por su tipo, tal y como serían aceptados por un READ.

(PRINT objeto &optional stream)

Ejemplos:

```
> (print 'a) ;           imprime A con #\Newline
> (print '(a b)) ;       imprime (A B) con #\Newline
> (print 99) ;           imprime 99 con #\Newline
> (print "hi") ;         imprime "hi" con #\Newline
```

PRIN1, toma un objeto Lisp como argumento y lo escribe con un blanco por detrás. Escribe los objetos por su tipo, tal y como serían aceptados por un READ, sin salto de línea. La próxima impresión será al lado.

(PRIN1 objeto &optional stream)

Ejemplos:

```
> (prin1 'a) ;           imprime A sin #\Newline
> (prin1 '(a b)) ;       imprime (A B) sin #\Newline
> (prin1 2.5) ;          imprime 2.5 sin #\Newline
> (prin1 "hi") ;         imprime "hi" sin #\Newline
```

PRINC, toma un objeto Lisp como argumento y lo escribe con un blanco por detrás, y en el caso de los string, imprime sin las comillas. No escribe los objetos por su tipo.

(PRINC objeto &optional stream)

Ejemplos:

```
> (princ 'a) ;           imprime A sin #\Newline
> (princ '(a b)) ;       imprime (A B) sin #\Newline
> (princ 99) ;           imprime 99 sin #\Newline
> (princ "hi") ;         imprime hi sin #\Newline
```

PPRINT, toma un objeto Lisp como argumento y lo escribe con un blanco por detrás, introduce solo un Return. No realiza la segunda evaluación.

(PPRINT objeto &optional stream)

Ejemplos:

```
> (pprint 'a) ;          imprime A returns NIL
> (pprint "abcd") ;      imprime "abcd" returns NIL
```

FORMAT, aparece como un mecanismo más generalizado de dar la salida. Se indican directivas que son como variables situadas entre el string y comienzan con ~. Muchas directivas miran el siguiente argumento de la lista del format y lo procesan de acuerdo a sus propias reglas. Su forma es

(FORMAT destino control-string &rest argumentos)

- destino = nil/ t/ stream. Si se indica el nil, se creará un string con las características indicadas y éste será lo que devuelva el format. Si se indica t, será la salida por defecto y sino será otro dispositivo indicado en stream.

- control-string = contiene el texto a escribir y "directivas"

- argumentos = lista con los parámetros para las "directivas"

Se usan diferentes directrices para procesar diferentes tipos de datos e incluirlos en el string:

~A - Imprime un objeto cualquiera.,
~D - Imprime un número en base 10
~S - Imprime una expresión simbólica.

Además, se puede utilizar el símbolo @ junto con la directiva ~A, para justificar a la derecha valores numéricos. No todas las directrices utilizan argumentos: ~

% - Inserta una nueva línea y
~| - Nueva página.

Ejemplos,

```
> (FORMAT T "STRING DE SALIDA")
STRING DE SALIDA
NIL
> (SETQ VAR 5)
5
> (FORMAT T "STRING QUE INCLUYE ~A" VAR)
STRING QUE INCLUYE 5
NIL
> (FORMAT NIL "STRING QUE INCLUYE ~A" VAR)
STRING QUE INCLUYE 5
> (FORMAT T "ESCRIBE EL ARGUMENTO ~10@A JUSTIFICADO A LA
DERECHA CON 10 ESPACIOS." 1000)
ESCRIBE EL ARGUMENTO      1000 JUSTIFICADO A LA DERECHA CON 10
ESPACIOS.
NIL
> (FORMAT T "~%LA PRIMERA RESPUESTA ES ~5@A ~%Y LA SEGUNDA
RESPUESTA ES ~3@A " (* 5 3 2) (/ 3 5))
LA PRIMERA RESPUESTA ES 30
Y LA SEGUNDA RESPUESTA ES 0.6
NIL
TERPRI realiza un salto de línea
(TERPRI &optional stream)
```

Ejemplos de salida:

```
> (print "hola") ==> hola
"hola"
> (print "hola") ==> "hola"
"hola"
> (prin1 "hola") ==> "hola"
"hola"
> (format t "hola") ==> hola
NIL
> (format nil "hola")
"hola"
> (format nil "hola soy ~a, ~a" "yo" "Bonn")
"hola soy yo, Bonn"
```

FUNCIONES DEFINIDAS POR EL USUARIO

LIGADURA, ASIGNACION Y AMBITO

Una función es un objeto Lisp que puede invocarse como un procedimiento. Una función puede tomar una serie de argumentos y devolver uno o más valores, pero al menos uno. Lisp tiene un gran número de funciones predefinidas, a menudo llamadas primitivas, aunque también permite la definición de funciones propias.

Lisp es un lenguaje funcional en el que todos los programas se escriben como colecciones de funciones y procedimientos. La definición de la función se almacena en memoria como un atributo/objeto del símbolo.

Por su parte una función se referenciará por el nombre imprimible de un símbolo cuando se realice una llamada a dicha función.

Por ejemplo, la llamada a la función de suma (+ 2 3).

La llamada a la función es una lista cuyo primer elemento es el nombre de la función y el resto formarán los parámetros de dicha función.

Lisp comprobará que efectivamente el primer elemento de la lista se corresponde con alguna función predefinida o del usuario. Los argumentos de la llamada a la función serán igualmente evaluados antes de pasárselos como información a la función.

Si el argumento de una función es a su vez otra llamada a función, entonces se evalúa primero la función del argumento y el resultado de la misma se pasa a la función principal.

Por ejemplo:

ENTRADA: (+ 3 (* 2 4))

EVALUACION DE ARGUMENTOS: (+ 3 8)

SALIDA: 11

DEFINICION DE FUNCIONES

DEFUN (Define FUNction), permite al usuario definir sus propias funciones. En general una función consta de tres partes:

nombre de la función, argumentos y cuerpo (lo que se suele llamar lambda expresión).

(DEFUN nombre-función lambda-lista {declaration: string-doc} cuerpo)

Donde nombre-función debe ser el nombre de un símbolo atómico, como para una variable, la lambda-lista es el conjunto de argumentos que se pueden pasar (hay varias opciones en las lambda-listas), de las declaraciones y documentación hablaremos un poco más adelante, y cuerpo es cualquier forma a evaluarse en secuencia cuando se invoca al nombre de la función.

Ejemplo,

```
> (defun PRIMERO (L)
```

```
(CAR L) ) ==> PRIMERO
```

```
> (PRIMERO '(A B C)) ==> A
```

```
> (defun RESTO (L)
```

```
(CDR L) ) ==> RESTO
```

```
> (RESTO '(A B C)) ==> (B C)
```

El parámetro L de la definición se corresponde con una variable local, es decir, sólo es accesible desde el interior de la función. Si hay una variable externa con igual nombre, la L que hace referencia dentro del cuerpo se refiere a la del argumento.

Las declaraciones se utilizan para proporcionar información extra al sistema. A menudo son mensajes utilizados por el compilador. Las declaraciones también pueden aparecer al principio del cuerpo de ciertas formas como en las funciones. Los string de documentación se utilizan para describir comandos y pueden imprimirse al invocar la función de documentación de la función. Ejemplo:

```
> (defun CUADRADO (x)
  "Esta función devuelve el cuadrado de un número"
  (* x x) ; fin de CUADRADO
)          ==> CUADRADO
```

Esta función utiliza un string de documentación y un comentario justo antes del último paréntesis de la función.

```
> (CUADRADO 3)
9
```

¿Qué sucede al definir la función CUADRADO?

- Lisp crea un símbolo CUADRADO
- Incorpora en la definición de función el valor correspondiente a dicha función.

Y ¿cómo se evalúa y activa una función?

La evaluación de una función, cuando esta se define, consiste en el nombre de la propia función. Por ejemplo, al definir en LISP la función CUADRADO, devuelve como valor de la misma CUADRADO.

```
>> (defun CUADRADO (x)
  (* x x) )          ==> CUADRADO
```

Por otro lado, cuando se llama a una función con ciertos argumentos, el valor que devuelve dicha función se corresponde con la última forma ejecutada en el cuerpo de DEFUN. En la función CUADRADO, corresponde con la forma (* x x)

Ejemplos

```
> (cuadrado 3)
9
> (cuadrado 2)
4
> (cuadrado (cuadrado 3))
81
> (cuadrado (+ -4 5 (/ 3 1)))
16
```

La definición de una función puede incluir tantos argumentos como se estimen necesarios. Por ejemplo:

```
> (defun SUMA-CUADRADO (x y)
  (+ (cuadrado x) (cuadrado y)) )
SUMA-CUADRADO
> (suma-cuadrado 2 3)
13
```

Se pueden definir también funciones sin parámetros:

```
> (defun SALUDO () (print "hola a todos") )
SALUDO
> (saludo)          ==> "hola a todos"
"hola a todos"
```

La respuesta de LISP al llamar a la función SALUDOS, es mediante dos strings que corresponderán: el primer string a la acción del comando PRINT, el segundo es el valor devuelto por la función saludo.

FUNCIONES ESPECIALES SIN NOMBRE O FUNCIONES LAMBDA

En algunas ocasiones, es deseable crear función es sin nombre. Estas funciones se conocen con el nombre de lambda-expresiones y tienen una estructura semejante a las de las funciones.

((lambda (p1 ... pn) cuerpo) '(a1 ... an))

Además, el comportamiento es semejante al de la función, con la diferencia que los a1...an ya son los parámetros pasados a la función. Ejemplos,

```
> ((lambda (x y) (cons (car x) (cdr y))) ) '(a b) '(c d))  
                                     ; '(a b) '(c d) son los parámetros pasados a la función.  
(a d)  
> ((lambda (a b) (* a b)) 4 8)      ; 4 y 8 son los parámetros pasados a la función  
32
```

LIGADURA DE PARAMETROS EN LA FUNCION

Las variables creadas en la lista de argumentos de una DEFUN (parámetros formales) son normalmente locales para la función. La lista de parámetros organiza un espacio en memoria para el ámbito de definición de la función, que será utilizado por los valores que se pasen a dicha función en la llamada.

```
> (defun mi-funcion (varx)  
    varx )                               ==> MI-FUNCION  
> (mi-funcion "vale")                   ==> "vale"  
> varx                                  ==> ERROR, UNBOUND VARIABLE
```

Esta variable varx sólo tiene sentido dentro de la función mi- funcion, no tiene acceso desde fuera y por tanto si pretendemos tener información de dicha variable nos dará un ERROR.

- La lista de parámetros apunta a los mismos valores que los correspondientes argumentos en la expresión de llamada.
- Cuando se termina la ejecución de la función se desligan los valores de la lista de parámetros.
- Las variables de la lista de parámetros no afectan a las variables que se encuentran fuera del procedimiento.
- Las variables ligadas, de la lista de argumentos de una función, no pueden ser accedidas por otras funciones llamadas en ejecución desde la misma (a no ser que estos valores se pasen como argumentos de la nueva función llamada).

FUNCIONES DE ASIGNACION

La función SETF, es utilizada como las SETQ para realizar asignación de valores a variables. Su sintaxis es

SETF {LUGAR FORM}+)

SETF utiliza la forma lugar para, a diferencia de SETQ, determinar una localización o dirección en la que hay que realizar la asignación. El lugar queda restringido a variables o alguna forma de función aceptable que normalmente evalúa un lugar al que asignar un valor.

Tal y como la hacía SETQ, realiza las evaluaciones y asignaciones secuencialmente, devolviendo el valor de la última evaluación realizada. Ejemplos,

```
> (SETF NOMBRE "MARIA")           ==> "MARIA"
```

```
> (SETF X '(A B C))               ==> (A B C)
```

```
> (SETF (CAR X) (CAR '(1 2 3)))   ==> 1
```

; acá se ve la utilidad de SETF, en el cual se reemplaza el contenido de una posición por el resultado de una función.

```
> X                               ==> (1 B C)
```

La Funcion Let permite crear variables locales interiores a una función. Su forma general consta de dos partes asignación de variables y cuerpo , y lo podemos expresar:

(LET ({VARIABLE}* / {VARIABLE FORMA}*) {DECLARATIONS}* CUERPO)

La asignación de variables es una lista de listas donde cada lista consta de un nombre de variable y una forma a evaluar. Cuando un LET es llamado las formas se evalúan asignando paralelamente valores a todas las variables. Si quisiéramos que las variables se asignen secuencialmente, se usa LET* en lugar de LET.

Por su parte el cuerpo del LET es como el cuerpo de un DEFUN. Cualquier valor previo que tuvieran variables con el mismo nombre se guardarán y se restaurarán al finalizar el LET.

Las operaciones incluidas en el LET son evaluadas en el entorno del LET.

Las variables creadas en el LET son locales (al LET) y por tanto no pueden ser accedidas desde el exterior (tal y como sucede con DEFUN).

Los parámetros del LET se ligan sólo dentro del cuerpo del LET. Las asignaciones de SET dentro de un parámetro LET no crean ni modifican variables globales. Las asignaciones realizadas por el SET en variables libres crearán o cambiarán valores globales. Ejemplos:

```
> (SETF X 5)
```

```
> (SETF E 0)
```

```
> (SETF TRIPLE 0)
```

```
> (LET ((D 2) (E 3) (CUADRUPLE))
```

```
    (SETF DOBLE (* D X))
```

```
    (SETF TRIPLE (* E X))
```

```
    (SETF CUADRUPLE (* 4 X)) )
```

```
20
```

```
> D
```

```
ERROR - UNBOUND VARIABLE      ; variable no ligada
```

```
> E
```

```
0
```

Observar que las variables D y E, están definidas en el ámbito del LET, pero además E, es una variable definida por SETF afuera del LET, por eso se devuelve su resultado.

```
> DOBLE
```

```
10
```

```
> TRIPLE
```

```
15
```

```
> CUADRUPLE
```

```
ERROR - UNBOUND VARIABLE
```

Observar que la variable CUADRUPLE está definida en el ámbito del LET, por lo tanto, es una variable local.

Los valores de las variables libres pueden modificarse en la ejecución del LET. Tal es el caso de la variable DOBLE.

Así como para evitar que en el LET se asignen las variables en paralelo se usa el LET*, en el caso del DO que también se asignan las variables en paralelo, se puede utilizar DO*.

DO* A diferencia del anterior realiza la asignación de variables secuencialmente.

```
> (defun exponente (m n)
  (do* ((resultado m (* m resultado))
        (exponente n (1- exponente))
        (contador (1- exponente) (1- exponente)) ) ;fin-pa
        ((zerop contador) resultado) ) ;fin-do ) ;fin-defun
```

EXPONENTE ; (ejemplo de la función exponente con DO*)

```
(defun exponencial-con-times (m n)
  (let ((resultado 1))
    (dotimes (cuenta n resultado)
      (setf resultado (* m resultado) ) ) ) )
```

exponencial-con-times

```
> (exponencial-con-times 4 3)          ==> 64
```

DECLARACIONES

Las variables, a menos que sean declaradas de otra forma, son de ámbito léxico. Lo que se ha visto hasta ahora con respecto a la ligadura para los argumentos de una función es que sólo pueden verse dentro del texto de dicha función (tienen ámbito léxico).

Las variables pueden declararse como especiales y en estos casos se dice que son de ámbito dinámico. Es decir, son variables libres que pueden ser accedidas desde cualquier parte del programa a partir del punto en el que han sido definidas.

Para declarar variables especiales para todas las funciones, sin necesidad de declararlas separadamente en cada una de ellas, usamos:

- DEFVAR y DEFPARAMETER
- DEFCONSTANT

Esta es la idea de variable global utilizada en otros lenguajes de programación. La forma sintácticamente correcta de las mismas es la siguiente:

- (DEFVAR nom-var [valor]), declara una variable global de nombre nom-var accesible desde cualquier parte del programa (desde cualquier función).
- (DEFPARAMETER nom-var [valor]), tiene las mismas características que DEFVAR, excepto que será usado como parámetro y no como variable.
- (DEFCONSTANT nom-var valor), el valor permanecerá constante en nom-var, si se intenta cambiar su valor dará error.

Ejemplo.

```
> (defconstant c 36)
```

```
c
```

```
> c
```

```
36
```

```
> (setf c 40)
```

Error: C has constant value 36 and cannot be changed to 40

RECURSIVIDAD EN LISP

Se dice que un proceso es recursivo cuando está basado en su propia definición. En programación, la recursividad, no es una estructura de datos, sino una técnica de programación que permite que un bloque de instrucciones se ejecute una n cantidad de veces. En muchas ocasiones reemplaza a las estructuras repetitivas.

En LISP, es común que las funciones se llamen a sí mismas en el cuerpo de la función, esto es lo que se llama una función recursiva.

CARACTERISTICAS DE LA PROGRAMACION RECURSIVA

- Implementación intuitiva y elegante.
- La traducción de la solución recursiva de un problema (caso base y caso recursivo) a código Lisp es prácticamente inmediata.
- Útil cuando hay varios niveles de anidamiento. La solución para un nivel es válida para el resto.
- La interpretación y comprensión del código puede ser compleja.

Ejemplos.

1. Función Factorial (n!)

```
(defun factorial (n)
  (if (= n 0) 1
      (* n (factorial (- n 1)))
  )
)
```

) ; existe una llamada a función factorial en el cuerpo de la función factorial.

2. Función Potencia (m^n)

```
(defun potencia (x m)
  (if (= m 0) 1
      (* x (potencia x (- m 1)))
  )
)
```

) ; existe una llamada a la función potencia desde el cuerpo de la función potencia.

3. Obtener el valor correspondiente a la posición n de una serie de Fibonacci.

Método Iterativo

```
(defun fibonacci (n)
  (do ((i 1 (1+ i))
      (fib1 0 fib)
      (fib 1 (+ fib fib1)))
      ((= i n) fib)))
```

Método Recursivo

```
(defun fibo-recur (x)
  (if (< x 2)
      x
      (+ (fibo-recur (1- x)) (fibo-recur (- x 2)))))
```

SEGUIMIENTO DE FUNCIONES (Trace)

Cuando se necesita saber exactamente cómo una función está trabajando en un punto particular en el código, hacer Break y usar el depurador de Lisp son herramientas

indispensables. Pero es trabajoso y lento (por lo menos en relación con la ejecución normal del programa).

A veces, es suficiente saber que una determinada función se ha llamado y devolvió un valor. TRACE nos brinda esta capacidad imprimiendo el nombre de la función y sus argumentos a la entrada, y el nombre de la función y de sus valores a la salida. Todo esto ocurre sin cambiar el código fuente de la función.

```
(trace [<sym>...])
```

Ejemplo

```
> (trace factorial fibo potencia)
```

```
(FACTORIAL FIBO POTENCIA)
```

La ejecución del trace permanecerá activa hasta que se ejecute la macro UNTRACE. Si se ejecuta sin argumentos, detendrá el trazo de todas las funciones.

```
(untrace [<sym>...])
```

Si se ejecuta la macro trace sin argumentos, mostrara una lista con todas las funciones que estan siendo trazadas en su ejecución.

```
(trace)
```

```
(FACTORIAL FIBO POTENCIA)
```

```
> (fibo 4)
```

```
Entering: FIBO, Argument list: (4)
```

```
Entering: FIBO, Argument list: (3)
```

```
Entering: FIBO, Argument list: (2)
```

```
Exiting: FIBO, Value: 1
```

```
Entering: FIBO, Argument list: (1)
```

```
Exiting: FIBO, Value: 1
```

```
Exiting: FIBO, Value: 2
```

```
Entering: FIBO, Argument list: (2)
```

```
Exiting: FIBO, Value: 1
```

```
Exiting: FIBO, Value: 3
```

```
3
```

STREAMS

Es un tipo de datos que mantiene la información sobre el dispositivo (fichero) al que está asociado. Hará que las E/S sean independientes del dispositivo.

Un stream puede estar conectado a un fichero o a un terminal interactivo.

OPEN Esta función devuelve un stream.

```
(OPEN nomfich &key :direction )
```

donde nomfich será un string/ pathname/ stream;

:direction permitirá señalar si el dispositivo va a ser de lectura-entrada, :input, salida-escritura, :output, o ambos, :io.

Ejemplo.

```
> (setq f (open 'mine :direction :output)) ; crea un archivo llamado
```

```
MINE
```

```
> (print "hola" f) ; retorna "hola"
```

```
> (print "Segundo hola" f) ; retorna "Segundo hola"
```

> (close f)	; archiva "hola" <newline>
> (setq f (open 'mine :direction :input))	; abre el archive como input
> (read f)	; lee y devuelve "hola"
> (read f)	; lee y devuelve "Segundo
hola"	
> (close f)	; cierra el archivo

Algunos predicados con streams serían:

(STREAMP Objeto), dará T si objeto es un stream
 (OUTPUT-STREAM-P stream), dará T si stream es de salida.
 (INPUT-STREAM-P stream), será T si stream es de entrada.

CLOSE, Cierra un stream, desaparece la ligadura entre el fichero y el stream (en caso de estar ligado a él).

(CLOSE stream)

Algunos streams estandar

standar-input

standar-output

error-output

terminal-io

Funciones de Entrada de STREAMS

READ-CHAR lee un carácter y lo devuelve como un objeto de tipo carácter. Su forma sintácticamente correcta es,
 (READ-CHAR &optional stream)

READ-LINE lee caracteres hasta encontrar un carácter #\newline. Y devuelve la línea como un string de caracteres sin #\newline ->
 (READ-LINE &optional input-stream)

PEEK-CHAR lee un carácter del stream de entrada sin eliminarlo del stream. La próxima vez que se acceda para leer se leerá el mismo carácter.
 (PEEK-CHAR &optional peek-type input-stream) Si peek-type es t se saltarán los caracteres en blanco. Si su valor es nil no.

Funciones de Manipulación de STREAMS

(RENAME-FILE fichero nuevo-nombre), cambia de nombre un fichero. Si fichero es un nombre de stream, el stream y el fichero a él asociado resultan afectados.

(DELETE-FILE fichero), borra un fichero. Si fichero es un nombre de stream, el stream y el fichero a él asociado resultan afectados.

(FILE-LENGTH stream-de-fichero), devuelve la longitud de un fichero. Si esta no puede ser determinada devolverá nil.

(LOAD fichero &key :verbose :print :if-does-not-exist), carga un fichero al interprete Lisp. A partir de ese momento el entorno Lisp contará con los objetos Lisp definidos en ese fichero.

Las claves incluidas en la forma LOAD corresponden con:

:verbose Si es t permitirá al cargar el fichero, sacar un comentario sobre el *standar-output* indicando las características del fichero que está siendo cargado.

:print Si es t se saca un valor de cada expresión cargada sobre *standar-output*.

:if-does-not-exist Si es nil devuelve nil; si es t el error correspondiente en caso de que lo haya.

DOCUMENTACION

Se puede hacer comentarios en cualquier lugar del programa siempre que se utilice el ";", de tal forma que después de la coma comienza el comentario. Por convenio se utilizará:

;;; 4 ";" y un blanco, al comienzo del fichero para indicar su contenido.

;;; 3 ";" y un blanco, para realizar comentarios sobre una función y aparecerá en la línea posterior a la de definición de función (cabecera).

;; 2 ";" y un blanco, se utiliza para comentar una línea de código. En la línea inmediatamente seguida a esta.

; sólo un ; y sin blanco, se utiliza para explicar una línea de código en la misma línea.

Ejemplo de un fichero comentado:

```
;;;
*****
;;; Fichero: Ejemplo-comentario.lisp
;;; Fecha-de-creación: 20/02/2014
;;; Modulo: Apuntes de lisp.
;;; Comentarios: Este fichero ha sido creado con fines didácticos.
;;; Autores: Los profesores de Paradigmas
;;; UNNE
;;;
*****
```

Ejemplo de un fichero comentado:

```
(defun pareja (x)
  ;; Esta función devuelve la pareja del personaje que se pasa como valor del parámetro x.
  (case x
    ;; se realiza la selección.
    ((Peter-pan) "Campanilla") ;Campanilla es una chica que vuela.
    ((Mortadelo) "Filemón") ;Ambos son agentes de la T.I.A.
    ( t "no tiene pareja")
  ) ;Fin de selección
  ) ;Fin de la función.
  ;; Esperamos que haya encontrado su pareja con esta función. Si no modifíquela siempre que lo crea necesario.
  ;; Fin del fichero.
```

TEMA 6

PROGRAMACION CONCURRENTE Y PARALELA INTRODUCCION.

CONCEPTOS Y DEFINICIONES

La programación a la que estamos más acostumbrados, la secuencial, estuvo en sus inicios fuertemente influenciada por las arquitecturas de único procesador, en las cuales disponíamos como características base:

- de un único procesador (CPU),
- de los programas y los datos que están almacenados en memoria RAM, y
- de los procesadores, que se dedican básicamente a obtener un determinado flujo de instrucciones desde la RAM, ejecutando una instrucción por unidad de tiempo

En este sentido los programas secuenciales son totalmente ordenados, éstos nos indican en qué orden serán ejecutadas las instrucciones.

Y una particularidad importante, los programas secuenciales son deterministas:

mediante una misma secuencia de datos de entrada, se ejecutará la misma secuencia de instrucciones y se producirá el mismo resultado (salvo errores de ejecución causados por causas externas).

Esta afirmación no es cierta para la programación concurrente.

Tradicionalmente, el software es escrito para ser computado en serie.

Esto significa:

- Es ejecutado en una única computadora con una única CPU
- El problema se divide en una sucesión de instrucciones
- Las instrucciones se ejecutan una después de otra
- Solo una instrucción puede ser ejecutada en un determinado instante de tiempo

Características de Programas Secuenciales

- Es el estilo de programación que corresponde al modelo conceptual de Von Neumann.
- Un programa secuencial tiene una línea simple de control de flujo.
- Las operaciones de un programa secuencial están ordenadas de acuerdo con un orden estricto.
- El comportamiento de un programa es solo función de las sentencias que lo componen y del orden en que se ejecutan.
- El tiempo que tarda en ejecutarse cada operación no influye en el resultado de un programa secuencial.
- La verificación de un programa secuencial es sencilla:
- Cada sentencia da la respuesta correcta.
- Las sentencias se ejecutan en el orden adecuado.

La **Programación concurrente** tiene sus raíces en los sistemas operativos y en la programación de sistemas (años 60's).

Sistemas operativos fueron organizados con una colección de procesos ejecutándose concurrentemente ("al mismo tiempo"), algunos se ejecutaban en el procesador principal y otros en los controladores llamados canales.

La administración de estos sistemas se hacía a bajo nivel, en 1972 apareció el lenguaje Pascal concurrente, era el primer lenguaje de alto nivel para este objetivo.

La concurrencia surge ante la necesidad de plantear soluciones con el máximo rendimiento.

Existen varias aplicaciones computacionales que requieren una gran velocidad de cálculo:

- visualización
- bases de datos distribuidas
- simulaciones
- predicciones científicas.

Dentro de las áreas afines de la concurrencia tenemos:

- sistemas distribuidos
- sistemas en tiempo real

Acontecimientos que impulsaron la programación concurrente:

- El concepto de hilo hizo que los programas puedan ejecutarse con mayor velocidad comparados con los procesos utilizados anteriormente.
- Los lenguajes orientados objetos como Java que daban mejor soporte a la programación con la inclusión de primitivas.
- La aparición de Internet dadas las aplicaciones como el navegador, el chat están programados mediante técnicas de concurrencia.

CONCEPTOS Y DEFINICIONES: CONCURRENCIA

«Es el conjunto de anotaciones y técnicas utilizadas para describir mediante programas el paralelismo potencial de los problemas, así como para resolver los problemas de comunicación y sincronización que se presentan cuando varios procesos que se reportan concurrentemente comparten recursos».

Una propiedad básica de este tipo de programación es el no determinismo, se desconoce ante un instante de tiempo que va ocurrir en el siguiente instante:

- En el caso de un único procesador se desconoce si después la ejecución de una instrucción específica habrá alguna interrupción para brindar el procesador a otro proceso.
- En el caso del sistema multiprocesador las velocidades de los procesadores no están sincronizadas, por lo que se desconoce a priori cuál procesador va ser el primero en ejecutar su siguiente instrucción.
- Por tanto es necesario disponer de un modelo abstracto que permita verificar y corregir los sistemas concurrentes.
- Cada problema concurrente presenta un tipo distinto de paralelismo, la implementación depende de la arquitectura.
- Si se requiere trabajar en un ambiente independiente de la arquitectura se debe plantear un modelo para verificar que sea correcto independientemente del hardware en el que se ejecute.

La programación concurrente nos permite desarrollar aplicaciones que pueden ejecutar múltiples actividades de forma paralela o simultánea.

La programación concurrente es necesaria por varias razones:

- Ganancia de procesamiento, ya sea en hardware multiprocesador o bien en un conjunto dado de computadoras. Mediante la obtención de ganancias en recursos, en capacidad de cómputo, memoria, recursos de almacenamiento, etc.
- Incremento del throughput de las aplicaciones (solapamiento E/S con cómputo).
- Incrementar la respuesta de las aplicaciones hacia el usuario, poder atender estas peticiones frente al cómputo simultáneo. Permitiendo a los usuarios y computadores, mediante las capacidades extras, colaborar en la solución de un problema.
- Es una estructura más apropiada para programas que controlan múltiples actividades y gestionan múltiples eventos, con el objetivo de capturar la estructura lógica de un problema.
- Da un soporte específico a los sistemas distribuidos.

Otro punto con cierta controversia, son las diferencias entre los términos de programación concurrente, distribuida, paralela, y multithread. Todos ellos son usados en mayor o menor medida en la construcción de sistemas distribuidos y/o paralelos.

Usaremos programación concurrente como marco global a los diferentes tipos de programación distribuida, paralela y multithread, tendencia que viene siendo habitual en la literatura del campo distribuido.

Aun así, hay una serie de diferencias que hay que puntualizar:

Concurrencia frente a paralelismo: en paralelismo hay procesamiento de cómputo simultáneo físicamente, en concurrencia el cómputo es simultáneo lógicamente, ya que el paralelismo implica la existencia de múltiples elementos de procesamiento (ya sean CPU, o computadores enteros) con funcionamiento independiente; mientras que la concurrencia no implica necesariamente que existan múltiples elementos de procesamiento, ya que puede ser simulada mediante las capacidades de multiprocesamiento del SO sobre la CPU.

Un caso típico es la programación multithread, múltiples hilos de ejecución en una misma aplicación compartiendo memoria, pudiendo ser esta ejecución paralela si se ejecuta en una máquina multiprocesador, o concurrente, lógicamente si se ejecuta en un solo procesador. Aun existiendo estas diferencias, en la práctica las mismas técnicas son aplicables a ambos tipos de sistemas.

CONCURRENCIA FRENTE A PARALELISMO

Concurrente: “Coincidencia, concurso simultáneo de varias circunstancias (RAE)”

- Es la capacidad de algunas de las tareas que resuelven un problema de poder ser realizadas en paralelo.
- Está presente en la naturaleza, la vida diaria, los sistemas de cómputo, etc.

Paralelo: “A la vez, a un mismo tiempo, simultáneamente (RAE)”

- Es la condición en que dos o más tareas que resuelven un problema se realizan exactamente al mismo tiempo.

Mientras la concurrencia es una característica de las tareas, el paralelismo se asocia a los recursos que se disponen para poder realizar esas tareas simultáneamente.

CONCEPTOS Y DEFINICIONES: Paralelo y Distribuido

La palabra **paralelismo** suele asociarse con altas prestaciones en cómputo científico en una determinada plataforma hardware, por ej. supercomputadores, aunque cada vez se desplaza más a plataformas basadas en clusters locales (computadoras enlazadas por redes de propósito general, o bien redes de alta velocidad optimizadas para procesamiento paralelo). Estas computadoras están físicamente en un mismo armario o sala de cómputo.

La palabra distribuido se ha referido típicamente a aplicaciones, ejecutándose en múltiples computadoras (normalmente se implica además heterogeneidad de recursos), que no tenían por qué estar físicamente en un mismo espacio. Este término también relaja su significado debido a la extensión de los modelos paralelos, a ambientes como multicluster (múltiples clusters enlazados), o grid, que, aun siendo generalmente considerados como paralelos, se acercan más a los sistemas distribuidos.

En general estos términos son ampliamente difundidos, y usados con significados diferentes, aunque las distinciones tienden a desaparecer, en especial cuando se consideran cada vez sistemas más potentes de multiprocesamiento, y la fusión de sistemas paralelos y distribuidos aumenta con entornos como grid. Llegando en la mayoría de los casos actuales a sistemas híbridos donde se integran los tres tipos de programación (o de sistemas).

Podemos asimismo partir de una definición de **sistema distribuido** de computación como: Una colección de elementos de cómputo autónomos, ejecutándose en uno o más computadores, enlazados por una red de interconexión, y soportados por diferentes tipos de comunicaciones que permiten observar la colección como un sistema integrado.

Los sistemas distribuidos operan sobre redes de computadoras, pero también se puede construir uno que tenga sus elementos funcionando en un computador simple multitarea. Además del clásico basado en redes, se pueden incluir los computadores paralelos, y especialmente los servidores en cluster, y otros ambientes como las redes wireless, y las redes de sensores.

Por otra parte, la computación grid abarca coordinación de recursos que no están sujetos a control centralizado mediante el uso de protocolos e interfaces abiertas, de propósito general, para proporcionar diferentes servicios.

Uno de los parámetros más interesantes a la hora de enfocar la programación concurrente sobre un determinado sistema distribuido es conocer los diferentes estilos arquitectónicos en los que se basa, así como su modelo de interacción y organización interna de los componentes.

La Computación Paralela es el uso simultáneo de múltiples recursos de cómputo para resolver un problema computacional. Esto es:

- Se ejecuta utilizando múltiples procesadores
- El problema es dividido en partes que pueden resolverse concurrentemente
- Cada parte es dividida en una serie de instrucciones, que a su vez se ejecutan simultáneamente en diferentes procesadores
- Debe emplearse un mecanismo de coordinación general

Procesamiento paralelo

Un programa concurrente puede ser ejecutado por:

- Multiprogramación: los procesos comparten uno o más Procesadores
- Multiprocesamiento: cada proceso corre en su propio procesador, pero con memoria compartida
- Procesamiento Distribuido: cada proceso corre en su propio procesador conectado a los otros a través de una red

CARACTERISTICAS DE LOS PROGRAMAS CONCURRENTES

- Son programas que tienen múltiples líneas de flujo de control.
- Las sentencias de un programa concurrente se ejecutan de acuerdo con un orden no estricto.
- La secuencialización de un programa concurrente es entre hitos o puntos de sincronización.
- Un programa concurrente se suele concebir como un conjunto de procesos que colaboran y compiten entre sí.

- Para validar un programa concurrente:
- Las operaciones se pueden validar individualmente si las variables no son actualizadas concurrentemente.
- El resultado debe ser independiente de los tiempos de ejecución de las sentencias.
- El resultado debe ser independiente de la plataforma en que se ejecuta.

APLICACIONES DE LOS PROGRAMAS CONCURRENTES

- Aplicaciones clásicas:
- Programación de sistemas multicomputadores.
- Sistemas operativos.
- Control y monitorización de sistemas físicos.
- Aplicaciones actuales:
- Servicios WEB.
- Sistemas multimedia.
- Cálculo numérico.
- Procesamientos entrada/salida.
- Simulación de sistemas dinámicos.
- Interacción operador/máquina (GUIs)
- Tecnologías de componentes.
- Sistemas embebidos.

VENTAJAS DE LA PROGRAMACION CONCURRENTES

- Proporciona el modelo más simple y natural de concebir muchas aplicaciones.
- Facilita el diseño orientado a objetos de las aplicaciones, ya que los objetos reales son concurrentes.
- Hace posible compartir recursos y subsistemas complejos.
- En sistemas monoprocesadores optimiza el uso de los recursos.
- Reduce tiempos de ejecución en plataformas multiprocesadoras.
- Facilita la realización de programas fiables replicando componentes y/o procesos.

CLASIFICACIONES ARQUITECTURALES

Para la concepción de sistemas concurrentes y sus paradigmas de programación, hay que tener en cuenta que tanto en el caso distribuido, como en el paralelo los sistemas disponen de múltiples procesadores, que son capaces de trabajar conjuntamente en una o varias tareas para resolver un problema computacional.

Existen muchas formas diferentes de construir y clasificar en diferentes taxonomías los sistemas distribuidos y paralelos.

Presentaremos varias clasificaciones basadas en diferentes conceptos.

Taxonomía de Flynn

Ésta es una clasificación de sistemas arquitecturales muy usada en paralelismo y en cómputo científico, que nos sirve para identificar tipos de aplicaciones y sistemas en función de los flujos de instrucciones (ya sean threads, procesos o tareas) y los recursos disponibles para ejecutarlos.

S I S D Single Instruction Stream Single Data Stream	S I M D Single Instruction Stream Multiple Data Stream
M I S D Multiple Instruction Stream Single Data Stream	M I M D Multiple Instruction Stream Multiple Data Stream

En función de los conjuntos de instrucciones y datos, y frente a la arquitectura secuencial que denominaríamos SISD (single instruction single data), podemos clasificar las diferentes arquitecturas paralelas (o distribuidas) en diferentes grupos:

- SISD (single instruction single data)
- SIMD (single instruction multiple data),
- MISD (multiple instruction single data) y,
- MIMD (multiple instruction multiple data),
 - SPMD (single program multiple data).
 - MIMD Shared Memory (SMP o MPP).
 - UMA
 - NUMA
 - MIMD Distributed Memory (Paso de Mensajes y MPP).

SIMD: un solo flujo de instrucciones es aplicado a múltiples conjuntos de datos de forma concurrente. Unos procesos homogéneos (con el mismo código) sincrónicamente ejecutan la misma instrucción sobre sus datos, o bien la misma operación se aplica sobre unos vectores de tamaño fijo o variable. El modelo es válido para procesamientos matriciales y vectoriales, siendo las máquinas paralelas con procesadores vectoriales un ejemplo de esta categoría.

MISD: el mismo conjunto de datos se trata de forma diferente por los procesadores. Son útiles en casos donde sobre el mismo conjunto de datos se deban realizar muchas operaciones diferentes. En la práctica no se han construido máquinas de este tipo por las dificultades en su concepción.

MIMD: paralelismo funcional y/o de datos. No sólo distribuimos datos, sino también las tareas a realizar entre los diferentes procesadores/nodos. Varios flujos (posiblemente diferentes) de ejecución son aplicados a diferentes conjuntos de datos. Esta categoría no es muy concreta, ya que existe una gran variedad de arquitecturas posibles con estas características, incluyendo máquinas con varios procesadores vectoriales o sistemas de centenares de procesadores o bien con unos pocos.

SPMD: (variante de MIMD): en paralelismo de datos, utilizamos mismo código con distribución de datos. Hacemos varias instancias de las mismas tareas, cada uno ejecutando el código de forma independiente. SPMD puede verse como una extensión de SIMD o bien una restricción del MIMD. A veces suele tratarse más como un paradigma de programación, en el cual el mismo código es ejecutado por todos los procesadores (u nodos) del sistema, pero en la ejecución se pueden seguir diferentes caminos en los diferentes procesadores.

Por control y comunicación (Variantes MIMD)

Otra extensión de las clasificaciones anteriores de los modelos arquitecturales es la basada en los mecanismos de control y la organización del espacio de direcciones en el caso de las arquitecturas MIMD, las de más amplia repercusión hoy en día (y que se ajusta bien a diferentes sistemas distribuidos):

1. MIMD con memoria compartida (shared memory, MIMD)
2. Sistemas MIMD con memoria distribuida (distributed memory, MIMD)

1. **MIMD** con memoria compartida o multiprocesadores: En este modelo, los procesadores comparten el acceso a una memoria común. Existe un único espacio de direcciones de memoria para todo el sistema, accesible a todos los procesadores. Los procesadores se comunican a través de esta memoria compartida. Los procesadores tienen el mismo espacio de acceso a los mismos.

Como ejemplo, se destaca a los sistemas SMP (symetric multiprocessing), refiriéndose a la combinación de MIMD y memoria (físicamente) compartida. La limitación principal es el número de procesadores dependiendo de la red de interconexión, siendo habitual sistemas entre 2 a 16 procesadores, incluidos en una misma máquina. Aunque también existe la opción, de unir algunos de estos sistemas (formando un sistema híbrido), en lo que se conoce como SPP (scalable parallel processing) o CLUMP (clusters of multiprocessors), para formar un sistema mayor.

Pueden ser de dos tipos:

Sistema fuertemente acoplado, el sistema ofrece un mismo tiempo de acceso a memoria para cada procesador. Este sistema puede ser implementado a través de un gran módulo único de memoria, o por un conjunto de módulos de memoria de forma que se pueda acceder a ellos en paralelo por los diferentes procesadores. Tenemos un acceso uniforme a memoria (UMA).

Sistema ligeramente acoplado, el sistema de memoria está repartido entre los procesadores, disponiendo cada uno de su memoria local. Cada procesador puede acceder a su memoria local y a la de los otros procesadores, pero el tiempo de acceso a las memorias no locales es superior a la de la local, y no necesariamente igual en todas. Tenemos un acceso no uniforme a la memoria (NUMA).

2. Sistemas MIMD con memoria distribuida (distributed memory, MIMD), o también llamados multicomputadores: En este modelo, cada procesador ejecuta un conjunto separado de instrucciones sobre sus propios datos locales. La memoria no centralizada está distribuida entre los procesadores (o nodos) del sistema, cada uno con su propia memoria local, en la que poseen su propio programa y los datos asociados. El espacio de direcciones de memoria no está compartido entre los procesadores. Una red de interconexión conecta los procesadores (y sus memorias locales), mediante enlaces (links) de comunicación, usados para el intercambio

de mensajes entre los procesadores. Los procesadores intercambian datos entre sus memorias cuando se pide el valor de variables remotas.

Una última variante MIMD con memoria físicamente distribuida y red de interconexión dedicada, especialmente diseñada, son los sistemas MPP (massive parallel processing). En éstos, el número de procesadores puede variar desde unos pocos a miles de ellos. Normalmente, los procesadores están organizados formando una cierta topología (tanto física como lógica, como: anillo, árbol, malla, hipercubo, etc.), disponiendo de una red de interconexión entre ellos de baja latencia y gran ancho de banda.

CARACTERÍSTICAS DESEADAS DE UNA APLICACIÓN PARALELA

- Alto Rendimiento
- Eficiencia
- Escalabilidad
- Transparencia
- Portabilidad
- Tolerante a fallos

Alto Rendimiento

Se intenta reducir el tiempo de ejecución de los programas:

- La mejor solución puede diferir totalmente de la sugerida por los algoritmos secuenciales existentes.
- Los aspectos independientes de la máquina tales como la concurrencia deben ser considerados tempranamente, y los aspectos específicos de la máquina se deben demorar.
- Algunos temas pueden ser manejados automáticamente por las herramientas de especificación/diseño o por el S.O.



Metodología de diseño de algoritmos paralelos



Enfoque metódico para maximizar el rango de opciones consideradas, brindar mecanismos para evaluar las alternativas, y reducir el costo de backtracking por malas elecciones.

Eficiencia

Se considera con respecto a:

- Uso de recursos de cómputo: fracción de tiempo en que los procesadores están siendo utilizados.
- Energía: potencia y energía (estática y dinámica) que se utilizan.

Escalabilidad

Es la capacidad de mantener/incrementar la eficiencia del programa en proporción al tamaño del problema (número de elementos de procesamiento).

Transparencia

Significa ocultar al usuario/programador detalles que dan funcionalidad al sistema. Puede ser con respecto:

- a la Ubicación → no distinguir entre recursos locales o remotos.
- a la Concurrencia → El usuario no “ve” la competencia y sincronización por los recursos.

- al Paralelismo y su implementación → Partir de la especificación del problema y no detenerse en el modo de descomponerlo en tareas o de ejecutarlo en múltiples procesadores → mayor abstracción → muy difícil obtener alta eficiencia.

Portabilidad

- La portabilidad de código se obtiene a través de estándares que permiten que el desarrollo de programas paralelos/distribuidos se ejecuten sobre una variedad de arquitecturas (ejemplos: OpenMP, MPI, Java)
- La portabilidad de performance es mucho más difícil de lograr → significa que en diferentes arquitecturas el programa logre un rendimiento o eficiencia similar.

Tolerancia a Fallos

Es la habilidad de recuperar el sistema desde fallos de sus componentes sin perder la estabilidad del sistema. Un problema con las comunicaciones, entre los procesadores, reinicio del sistema, etc. no debería afectar el funcionamiento del sistema.

Modelos de Programación (Paralela/Distribuida)

Son formas de estructurar un algoritmo seleccionando una técnica de descomposición y mapeo en búsqueda de minimizar interacciones entre procesos.

Para los modelos de programación, remarcamos cómo solucionan la distribución de código y la interconexión entre las unidades de ejecución y las tareas. Cualquiera de estos modelos de programación puede ser usado para implementar los paradigmas de programación.

Pero las prestaciones de cada combinación resultante (paradigma en un determinado modelo) dependerán del modelo de ejecución subyacente (la combinación de hardware, red de interconexión y software de sistema disponibles).

Modelo de MEMORIA COMPARTIDA

En este modelo los programadores ven sus programas como una colección de procesos accediendo a variables locales y un conjunto de variables compartidas. Cada proceso accede a los datos compartidos mediante una lectura o escritura asíncrona. Por tanto, como más de un proceso puede realizar las operaciones de acceso a los mismos datos compartidos en el mismo tiempo, es necesario implementar mecanismos para resolver los problemas de exclusiones mutuas que se puedan plantear, mediante mecanismos de semáforos o bloqueos.

La aplicación se ve como una colección de tareas que normalmente son asignadas a threads de ejecución en forma asíncrona.

OpenMP es una de las implementaciones más utilizadas para programar bajo este modelo en sistemas de tipo SMP (o SH-MIMD).

OpenMP (open specifications for multi processing) define directivas y primitivas de librería para controlar la paralelización de bucles y otras secciones de un código en lenguajes como Fortran, C y C++. La ejecución se basa en la creación de thread principal, juntamente con la creación de threads esclavos cuando se entra en una sección paralela. Al liberar la sección, se reasume la ejecución secuencial. OpenMP, que normalmente se implementa a partir librerías de bajo nivel de threads.

En OpenMP se usa un modelo de ejecución paralela denominado fork-join, básicamente el thread principal, que comienza como único proceso, realiza en un momento determinado una operación fork para crear una región paralela (directivas PARALLEL, END PARALLEL) de un conjunto de threads, que acaba mediante una sincronización por una operación join, reasumiéndose el thread principal de forma secuencial, hasta la siguiente región paralela.

Todo el paralelismo de OpenMP se hace explícito mediante el uso de directivas de compilación que están integradas en el código fuente (Fortran, o C/C++).

Todos los datos de la aplicación están en una memoria global a la que puede acceder cada tarea de forma independiente.

Se requiere sincronización para preservar la integridad de las estructuras de datos compartidas.

Modelo de MEMORIA DISTRIBUIDA (PASO DE MENSAJES)

Éste es uno de los modelos más ampliamente usado. En él los programadores organizan sus programas como una colección de tareas con variables locales privadas y la habilidad de enviar, y recibir datos entre tareas por medio del intercambio de mensajes. Definiéndose así por sus dos atributos básicos: Un espacio de direcciones distribuido y soporte únicamente al paralelismo explícito.

Los mensajes pueden ser enviados vía red o usando memoria compartida si está disponible. Las comunicaciones entre dos tareas ocurren a dos bandas, donde los dos participantes tienen que invocar una operación. Podemos denominar a estas comunicaciones como operaciones cooperativas, ya que deben ser realizadas por cada proceso, el que tiene los datos y el proceso que quiere acceder a los datos. En algunas implementaciones, también pueden existir comunicaciones de tipo one- sided, si es sólo un proceso el que invoca la operación, colocando todos los parámetros necesarios y la sincronización se hace de forma implícita.

Ventajas: el paso de mensajes permite un enlace con el hardware existente, ya que se corresponde bien con arquitecturas que tengan una serie de procesadores conectados por una red de comunicaciones.

- En cuanto a la funcionalidad, incluye una mayor expresión disponible para los algoritmos concurrentes, proporcionando control no habitual en el paralelismo de datos, o en modelos basados en paralelismo implícito por compilador.
- En cuanto a prestaciones, especialmente en las CPU modernas, el manejo de la jerarquía de memoria es un punto a tener en cuenta; en el caso del paso de mensajes, deja al programador la capacidad de tener un control explícito sobre la localidad de los datos.

Desventaja, el principal problema del paso de mensajes es la responsabilidad que el modelo hace recaer en el programador.

TEMA 7

PROGRAMACION CONCURRENTES Y PARALELA.

Descomposición de Problemas y Especificación Formal

DESCOMPOSICION DE PROBLEMAS

La descomposición de los problemas a resolver plantea diferentes retos a nivel distribuido y paralelo, las aplicaciones distribuidas y/o paralelas consisten en una o más tareas que pueden comunicarse y cooperar para resolver un problema.

Las estrategias más comúnmente usadas incluyen:

- Descomposición funcional
- Descomposición geométrica

- Descomposición iterativa

Descomposición funcional: se rompe el cómputo en diferentes subcálculos, que pueden: a) realizarse de forma independiente; b) en fases separadas (implementándose en pipeline); c) con un determinado patrón jerárquico o de dependencias de principio o final entre ellos.

Descomposición geométrica: el cálculo se descompone en secciones que corresponden a divisiones físicas o lógicas del sistema que se está modelando. Para conseguir un buen balanceo, estas secciones deben ser distribuidas de acuerdo a alguna regla regular o repartidas aleatoriamente. Normalmente, es necesario tener en cuenta cierta ratio de cómputo-comunicación, para realizar un balanceo más o menos uniforme.

Descomposición iterativa: romper un cómputo en el cual una o más operaciones son repetidamente aplicadas a uno o más datos, ejecutando estas operaciones de forma simultánea sobre los datos. En una forma determinística, los datos a procesar son fijos, y las mismas operaciones se aplican a cada uno. En la forma especulativa, diferentes operaciones son aplicadas simultáneamente a la misma entrada hasta que alguna se complete.

En cuanto a la creación de las aplicaciones distribuidas o paralelas, basándose en las posibles descomposiciones, no hay una metodología claramente establecida ni fija, debido a la fuerte dependencia de las arquitecturas de las máquinas que se usen, y los paradigmas de programación usados en su implementación.

Una metodología básica

Una metodología simple de creación de aplicaciones paralelas y/o distribuidas podría ser la que estructura el proceso de diseño en cuatro etapas diferentes: partición, comunicación, aglomeración y mapping (a veces a esta metodología se la denomina con el acrónimo PCAM).

Las dos primeras etapas se enfocan en la concurrencia y la escalabilidad, y se pretende desarrollar algoritmos que primen estas características. En las dos últimas etapas, la atención se desplaza a la localidad y las prestaciones ofrecidas.

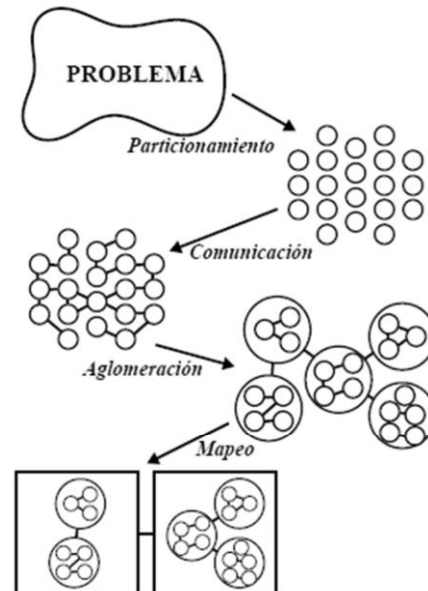
Partición: el cómputo a realizar y los datos a operar son descompuestos en pequeñas tareas. El objetivo se centra en detectar oportunidades de ejecución concurrente. Para diseñar una partición, observamos los datos del problema, determinamos particiones de estos datos y, finalmente, se asocia el cómputo con los datos. A esto se denomina descomposición del dominio. Una alternativa consiste en la descomposición funcional, asignando diferentes cómputos o fases funcionales a las diferentes tareas. Las dos son complementarias que pueden ser aplicadas a diversos componentes o fases del problema, o al mismo problema para obtener algoritmos distribuidos o paralelos alternativos.

Comunicación: se determinan las comunicaciones necesarias (en forma de estructuras de datos necesarias, protocolos, y algoritmos), para coordinar la ejecución de las tareas.

Aglomeración: las tareas y estructuras de comunicación de las dos primeras fases son analizadas respecto de las prestaciones deseadas y los costes de implementación. Si es necesario, las tareas son combinadas en tareas mayores, si con esto se consigue reducir los costes de comunicación y aumentar las prestaciones.

Mapping o mapeo: cada tarea es asignada a un procesador/nodo, de manera que se intentan satisfacer los objetivos de maximizar la utilización del procesador/nodo, y minimizar los costes de comunicación. El mapping puede especificarse de forma estática, o determinarlo en ejecución mediante métodos de balanceo de carga.

- ▶ **Particionamiento:** descomposición en tareas.
- ▶ **Comunicación:** estructura para coordinar la ejecución.
- ▶ **Aglomeración:** evaluación de tareas y estructura con respecto a rendimiento y costo, combinando tareas para mejorar.
- ▶ **Mapeo:** asignación de tareas a procesadores.



Mapeo de procesos

- Normalmente tendremos más tareas (procesos) que procesadores físicos.
- Los algoritmos paralelos (o el scheduler de ejecución) deben proveer un mecanismo de “mapping” entre tareas y procesadores físicos, esto significa especificar dónde se ejecuta cada tarea.
- Este problema no existe en uniprosesadores o máquinas de memoria compartida con scheduling de tareas automático.
- El lenguaje de especificación de algoritmos paralelos debe poder indicar claramente las tareas que pueden ejecutarse concurrentemente y su precedencia/prioridad para el caso que no haya suficientes procesadores para atenderlas.

Decidir el mapping es complicado porque:

- Ubicar tareas concurrentes en distintos procesadores ⇒ mejora la concurrencia, pero...
- Ubicar tareas que se comunican con frecuencia en el mismo procesador ⇒ mejora la localidad de los datos

Un buen mapping es crítico para la performance de los algoritmos paralelos. Debe encontrarse un equilibrio que optimice el rendimiento paralelo.

La clave es obtener un mapping que logre un balance para buscar el rendimiento de toda la arquitectura paralela.

Efecto de la Granularidad en la Performance

En algunas ocasiones, disminuir el número de procesadores puede mejorar la performance de un sistema paralelo:

- Se denomina scaling down de un sistema paralelo.

Un modo simple de realizar el scaling down es pensar que sobre un procesador físico real asignamos más de un procesador virtual. Si por ejemplo tenemos P procesadores reales y N procesadores virtuales:

N/P es el factor de aumento de los cómputos a realizar en cada procesador físico

ESPECIFICACION FORMAL DE SISTEMAS CONCURRENTES

¿Qué se puede ejecutar concurrentemente?

Ahora es necesario, dado un programa concurrente, saber que secciones del código son concurrentes y cuáles no, además es indispensable especificarlo en un lenguaje de programación

No todas las partes se pueden ejecutar en forma concurrente.

Considerando el siguiente fragmento de programa:

$x:=x+1; y:=x+2;$

Está claro que la primera sentencia debe ejecutarse antes que la segunda, sin embargo si consideramos ahora:

$x:=1; y:=2; z:=3;$

Se puede observar que el orden en que se ejecuten no interviene en el resultado final. Si se dispusiera de 3 procesadores, se podría ejecutar cada una de las líneas en uno de ellos, incrementando la velocidad del sistema.

Bernstein, definió unas condiciones para determinar si dos conjuntos de instrucciones S_i y S_j se pueden ejecutar concurrentemente.

Condiciones de Bernstein: Para poder determinar si dos conjuntos de instrucciones se pueden ejecutar de forma concurrente, se define en primer lugar los siguientes conjuntos:

- $L(S_k) = \{a_1, a_2, \dots, a_n\}$, como el conjunto de lectura del conjunto de instrucciones S_k y que esta formado por todas las variables cuyos valores son referenciados (se leen) durante la ejecución de las instrucciones en S_k .
- $E(S_k) = \{b_1, b_2, \dots, b_m\}$, como el conjunto de escritura del conjunto de instrucciones S_k y que esta formado por todas las variables cuyos valores son actualizados (se escriben) durante la ejecución de las instrucciones en S_k .

Para que dos conjuntos de instrucciones S_i y S_j se puedan ejecutar concurrentemente, se tiene que cumplir que:

- $L(S_i) \cap E(S_j) = \emptyset$
- $E(S_i) \cap L(S_j) = \emptyset$
- $E(S_i) \cap E(S_j) = \emptyset$

(L=Lectura, E=Escritura)

Como ejemplo supongamos que tenemos:

$S1 \rightarrow a := x + y;$

$S2 \rightarrow b := z - 1;$

$S3 \rightarrow c := a - b;$

$S4 \rightarrow w := c + 1;$

Empleando las condiciones de Bernstein veremos que sentencias pueden ejecutarse de forma concurrente y cuáles no. Para ello, vamos a establecer los conjuntos de lectura y escritura correspondientes:

$$\begin{array}{ll}
 L(S_1) = \{x,y\} & E(S_1)=\{a\} \\
 L(S_2) = \{z\} & E(S_2)=\{b\} \\
 L(S_3) = \{a,b\} & E(S_3)=\{c\} \\
 L(S_4) = \{c\} & E(S_4)=\{w\}
 \end{array}$$

Luego se aplica las condiciones de Bernstein a cada par de sentencias:

Entre $S_1 \cap S_2$:

1. $L(S_1) \cap E(S_2)=\emptyset$
2. $E(S_1) \cap L(S_2)=\emptyset$
3. $E(S_1) \cap E(S_2)=\emptyset$

Entre $S_1 \cap S_3$:

1. $L(S_1) \cap E(S_3)=\emptyset$
2. $E(S_1) \cap L(S_3)=a \neq \emptyset$
3. $E(S_1) \cap E(S_3)=\emptyset$

Entre $S_1 \cap S_4$:

1. $L(S_1) \cap E(S_4)=\emptyset$
2. $E(S_1) \cap L(S_4)=\emptyset$
3. $E(S_1) \cap E(S_4)=\emptyset$

De todo se deduce la siguiente tabla en la que puede verse que pares de sentencias pueden ejecutarse en forma concurrente:

	S_1	S_2	S_3	S_4
S_1	-	Si	No	Si
S_2	-	-	No	Si
S_3	-	-	-	No
S_4	-	-	-	-

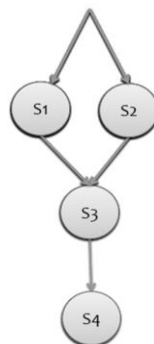
Una vez identificado que se puede o no ejecutar concurrentemente se hace necesario algún tipo de notación para especificarlas.

Veremos 2 formas de especificar la ejecución concurrente de instrucciones basadas en una notación gráfica «grafo de precedencia» y otra basada en lo que suelen utilizar diversos lenguajes de programación, el «par cobegin/coend».

Grafos de precedencia:

- Se trata de una notación gráfica.
- Es un grafo dirigido acíclico.
- Cada nodo representará una parte (conjunto de instrucciones) de nuestro sistema.
- Una flecha desde A hasta B, representa que B solo puede ejecutarse cuando A haya finalizado.
- Si aparecen dos procesos en paralelo, querrá decir que se pueden ejecutar concurrentemente.

Para el ejemplo anterior, el grafo de precedencia sería la siguiente figura:



Sentencias COBEGIN-COEND:

- Todas las acciones que puedan ejecutarse concurrentemente las instrucciones dentro del par cobegin/end.

- Las instrucciones en el bloque pueden ejecutarse en cualquier orden, el resto de manera secuencial.
- El ejemplo anterior quedaría:

- $S_1 \rightarrow a := x + y;$
- $S_2 \rightarrow b := z - 1;$
- $S_3 \rightarrow c := a - b;$
- $S_4 \rightarrow w := c + 1;$

```
Begin
cobegin
  a:=x+y
  b:=z-1
coend
c:=a-b;
w:=c+1;
end
```